Lab 05
Shell Scripting

Course: Operating System **Class: AI Section A and B.**

Instructor: Bilal Ahmed

Learning Objectives:

- Understanding shell and its types.
- Understanding Bash and Sh.
- Basic scripts and syntax.
- Operators in bash.
- Understanding .bashrc file.

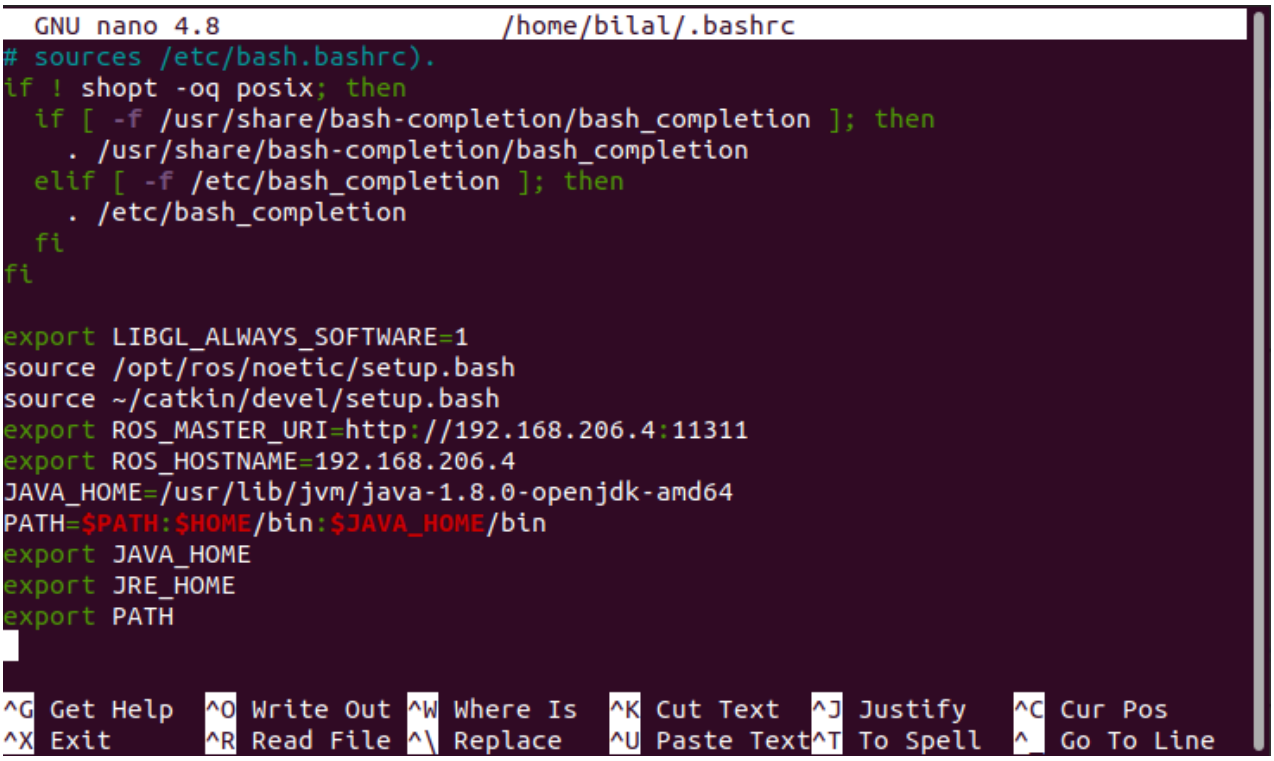
Introduction: Shell scripting in Linux streamlines task execution by allowing users to write sequences of commands in script files. These scripts automate repetitive tasks, saving time and effort. They can be executed manually or scheduled for automatic execution, enhancing productivity. Scripts can also be configured to run on system startup, performing tasks like displaying messages or setting environment variables. Beyond executing commands, the shell acts as both an interactive environment and an interpreter, providing a versatile tool for organizing and automating workflows within the Linux ecosystem. Following are the well-known types of Linux shells.

- **Bourne Shell (sh):** The original Unix shell, providing a simple command-line interface and scripting capabilities.
- **C Shell (csh):** A Unix shell with C-like syntax and interactive features, designed for interactive use rather than scripting.
- **TENEX C Shell (tcsh):** An enhanced version of the C Shell, offering additional features such as command-line editing and history.
- **KornShell (ksh):** A Unix shell developed by David Korn, combining features of the Bourne Shell and the C Shell with advanced scripting capabilities.
- **Debian Almquist Shell (dash):** A lightweight POSIX-compliant shell designed for efficiency and speed, commonly used as the default /bin/sh on Debian-based systems.
- **Bourne Again Shell (bash):** The default shell for most Linux distributions, featuring extensive scripting capabilities, command-line editing, and interactive features.
- **Z Shell (zsh):** An extended Bourne-compatible shell with additional features like advanced tab completion, improved scripting capabilities, and customizable prompts.

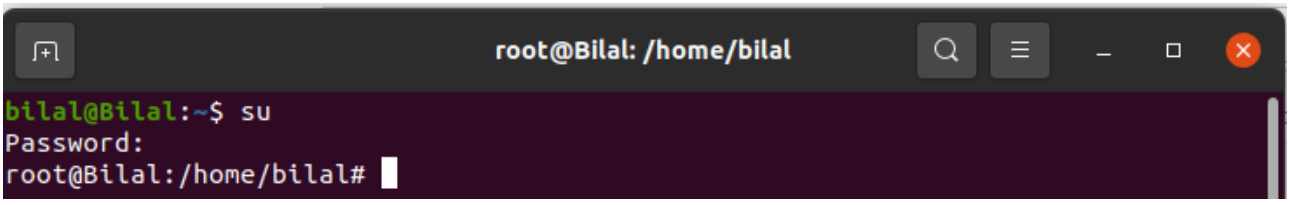
- **Friendly Interactive Shell (fish):** A user-friendly shell with intuitive syntax highlighting, autosuggestions, and powerful scripting capabilities, aiming for simplicity and ease of use.

Bash: Like Ubuntu many Linux distros use bash as default shell which is succeeder of sh. Uses also can configure system to use another shell.

Bashrc: A start-up script which Linux systems possess which is loaded every time we open the terminal. And it can be modified to have customized shell response. As we can see the necessary variables in the bashrc file that are loaded every time

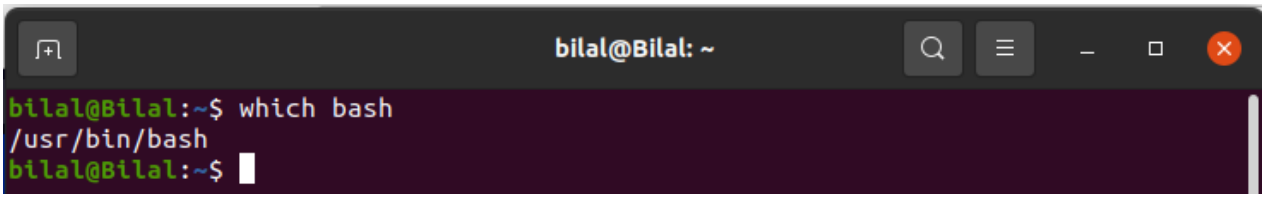


Shell Modes: When a shell is in interactive mode, it displays $. It means shell is waiting for a user to pass a command. If a shell is being used as a root it shows # as illustrated below.

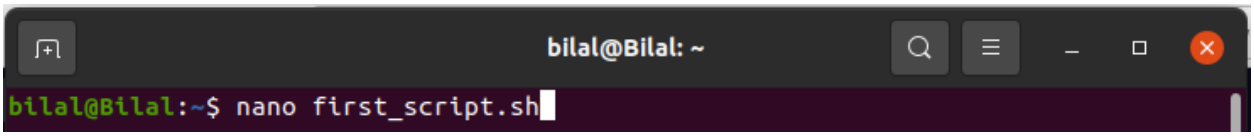


Bash Script: Sequence of commands to do a particular task which will be executed using bash program. For example, to create a folder with two python files. To do this we need multiple commands which we can save in bash program and run it.

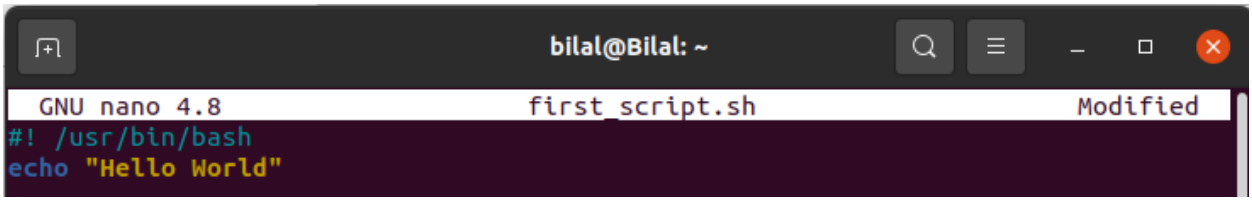
Shebang: The shebang directs the system to execute a script using the bash shell. It consists of the absolute path to the bash interpreter. To know the shell path just type which bash in the terminal as below.



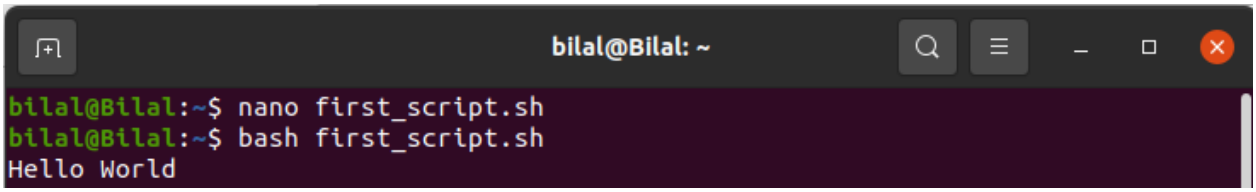
First Bash Script: To create a bash script, we can use any editor as shown below using nano.



Script: Add following lines.

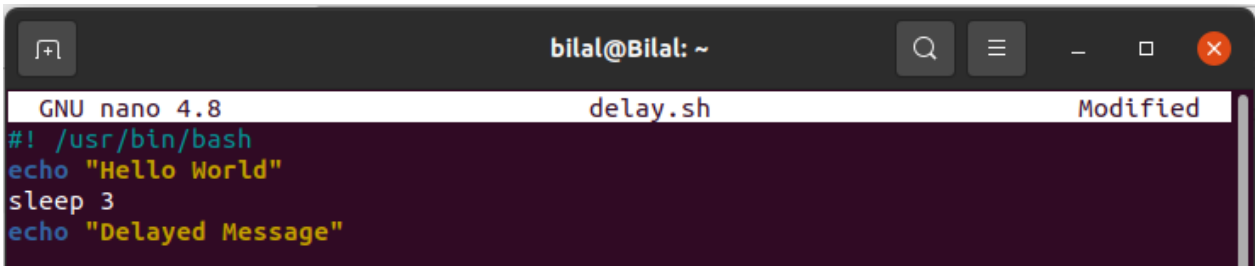


Executing the script (Method 1): To execute the script just execute it with bash command as given below.



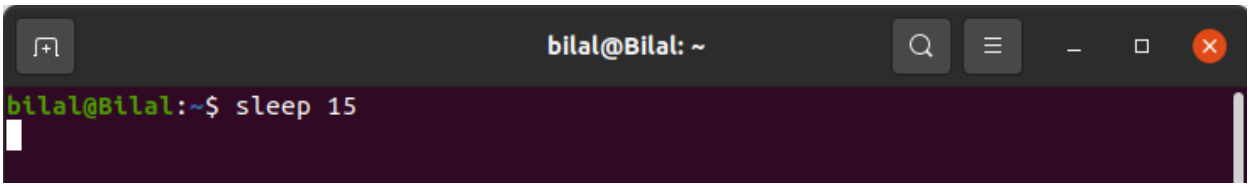
Executing the script (Method 2): We can change its permissions and then run it with ./script.sh as given below.

Delay in shell script: We can introduce delay between the execution of the statements using sleep command as given below.

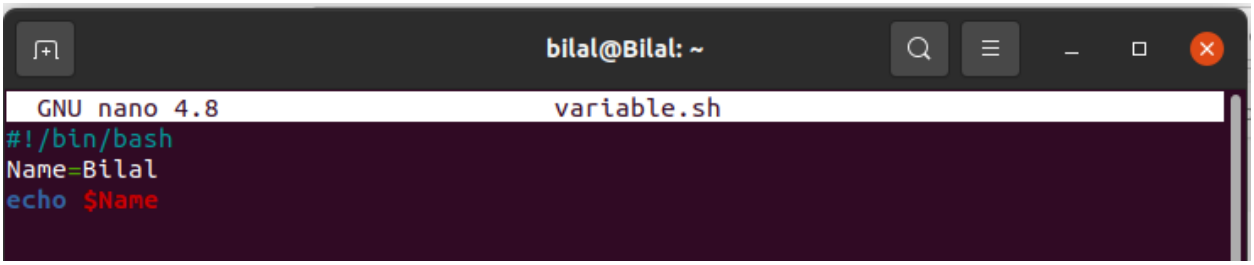


Output: Below we will get the delayed message after 3 seconds of execution.

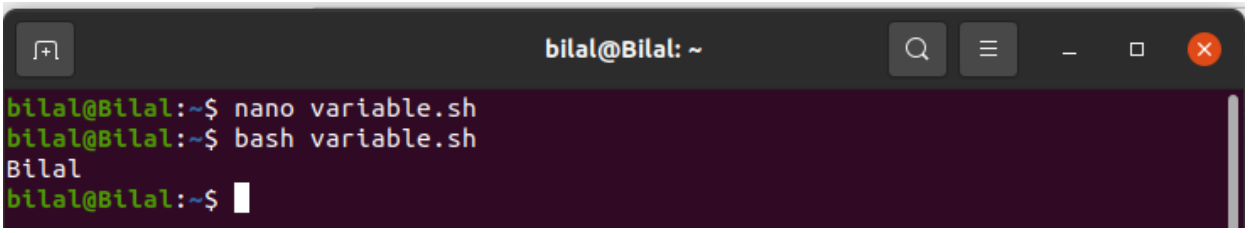
Executing sleep command in interactive mode:



Variables: We can define variables in scripts similar to programming languages as below. The $ symbol is used here to access the content of variable.



Output: We can see that content of variable is printed out.



Multiple Variables: We can use multiple variables similarly as given below.

```
bilal@Bilal: ~
 GNU nano 4.8                    variable.sh
#!/bin/bash
Name=Bilal
Surname=Arain
echo $Name $Surname
```

Output:

```
bilal@Bilal: ~
bilal@Bilal:~$ bash variable.sh
Bilal Arain
```

Use Case 1: We will write a simple script which will navigate catkin/src directory, will list the files present in the directory and will create a python file as below given. All steps will have 3 seconds delay.

Note: Create a sample directory and put some random files.

```
bilal@Bilal: ~
 GNU nano 4.8                    use_case1.sh                    Modified
echo "Navigating to catkin source direcotry ....."
sleep 3
cd ~/catkin/src

echo "Listing the content present in current directroy ....."
sleep 3
ls

echo "creating folder ....."
sleep 3
touch use_case1.py
```

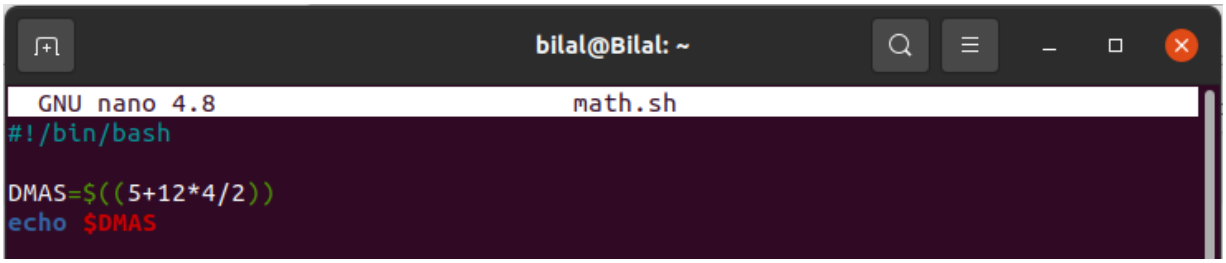
```
bilal@Bilal: ~
bilal@Bilal:~$ bash use_case1.sh
Navigating to catkin source direcotry .....
Listing the content present in current directroy .....
CMakeLists.txt  differential-drive  navigation  Pakbot  use_case1.py
creating folder .....
```

Mathematical Operations: Numeric expressions can be evaluated and assigned to a variable using the following syntax.

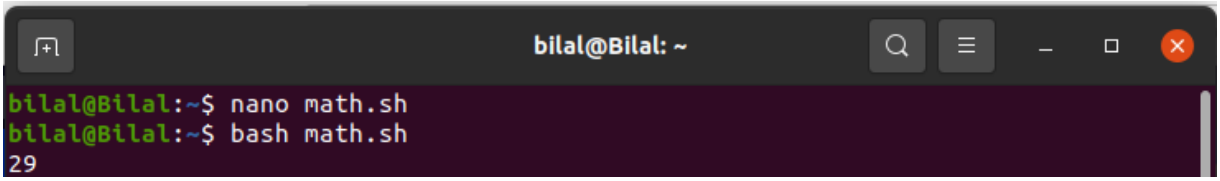
```
bash                                                    Copy code

var=$((expression))
```

Example: Let's try a simple addition example of addition.

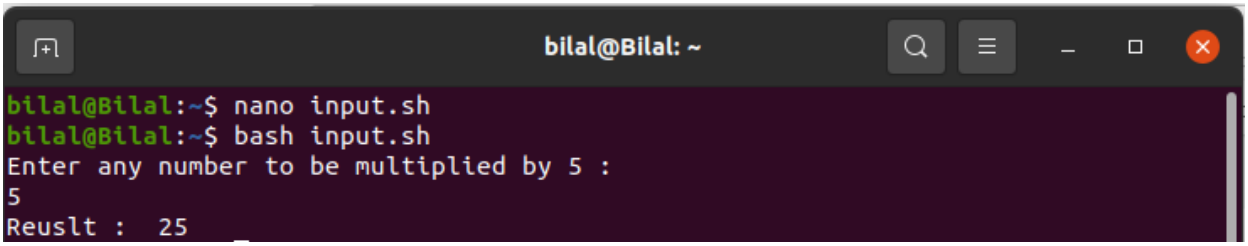


Output:



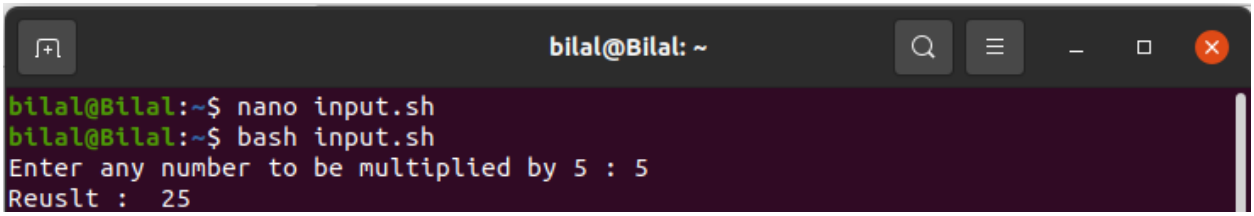
User Input: We can take user input using read command as below.

Output: Below we can see the output is 25 as input was 5.



Input with a custom message: We can get input with custom message using **–p option** without using echo as given below.

Output:



Use Case 2: Below script will ask user to enter the folder path to create a new folder and 3 files in the created folder. P, nf, f1, f2, and f3 variables are created for folder path, new folder name, file 1, file 2 and file 3 respectively. $ symbol is used to access the content of the variables and are assigned to respective places for navigation and creation.

Output: Below is the output of above script in which I have asked to navigate to **~/catkin folder**. After navigating I created a folder with the name **use_case2,** and in the respective folder I created 3 files with names as **f1.py f2.c and f3.java.**

Comparision operators: Similar to operators in traditional programming, bash has following options.

-eq: Checks if num1 is equal to num2.
-ge: Checks if num1 is greater than or equal to num2.
-gt: Checks if num1 is greater than num2.
-le: Checks if num1 is less than or equal to num2.
-lt: Checks if num1 is less than num2.
-ne: Checks if num1 is not equal to num2.

If Condition syntax:

bash Copy code

```bash
if [ condition ]
then
    # commands to execute if condition is true
elif [ condition ]
then
    # commands to execute if condition is true for elif
else
    # commands to execute if all conditions are false
fi
```

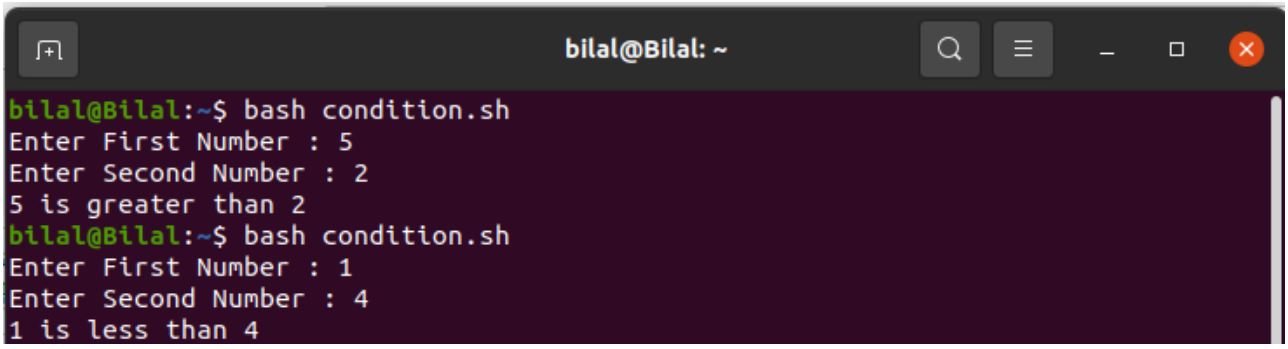
Example: Let's compare two numbers with each other.

```
 GNU nano 4.8                    condition.sh                         Modified
#!/bin/bash

read -p "Enter First Number : " x
read -p "Enter Second Number : " y

if [ $x -gt $y ]
then
echo $x is greater than $y
else
echo $x is less than $y
fi
```

Output:



Loop: We can also use loops in bash to iterate over the files and folders. The general syntax of the for loop is given below.

```bash
for variable in list
do
    # commands to execute
done
```

Example: Here is a simple example of a for loop to print values from 1 to 5.

Output:

File Operations: In order to perform file system operations, Bash provides the following file test operators with their respective symbols, syntax, and descriptions.

- **-d:** [-d DIRECTORY]: Checks if DIRECTORY exists and is a directory.
- **-f:** [-f FILE]: Checks if FILE exists and is a regular file (not a directory or a special file).
- **-e:** [-e FILE]: Checks if FILE exists.
- **-r:** [-r FILE]: Checks if FILE is readable.
- **-w:** [-w FILE]: Checks if FILE is writable.
- **-x:** [-x FILE]: Checks if FILE is executable.
- **-s:** [-s FILE]: Checks if FILE exists and has a size greater than zero (i.e., is not empty).
- **-L:** [-L FILE]: Checks if FILE exists and is a symbolic link.
- **-p:** [-p FILE]: Checks if FILE exists and is a named pipe (FIFO).
- **-S:** [-S FILE]: Checks if FILE exists and is a socket.

Use Case 3: This script will check whether the content is folder or a file and will add prefixes accordingly before the name of the file.

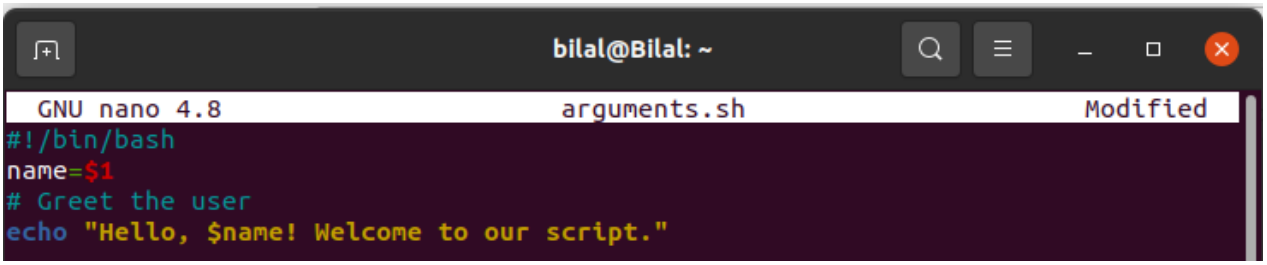
```
 GNU nano 4.8                    files.sh                         Modified
#!/bin/bash

# List all items in the current directory
for item in *
do
    # Check if the item is a directory
    if [ -d $item ]
    then
        echo Folder: $item
    # Check if the item is a regular file
    else
        echo File: $item
    fi
done
```

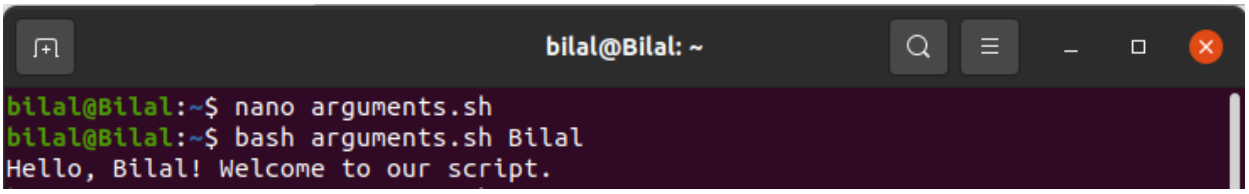
Output:

```
bilal@Bilal: ~
bilal@Bilal:~$ nano files.sh
bilal@Bilal:~$ bash files.sh
Folder: ,
File: addHosts.sh
Folder: Arduino
File: camera.py
File: capiutils_3.25+dfsg1-9ubuntu3_amd64.deb
File: Captured.jpg
Folder: catkin
Folder: cloudsim
Folder: CloudSimEx
File: cmd.py
File: condition.sh
```

Arguments in Bash: We can also pass arguments while running script in bash. To assign arguments we use numbers in bash as demonstrated below.



Output:



Editing Bashrc File: As discussed variables set in this file are called upon opening of a every new terminal. It is used for the following reasons.

- **Customization:** .bashrc customizes the shell environment with variables, aliases, and options.
- **Initialization:** It's executed at each new interactive shell session, ensuring user settings are applied.
- **Configuration:** Users set preferences like default editor, prompt format, and feature toggles.
- **Modularity:** Configurations can be modularized for organization and management.
- **Compatibility:** .bashrc ensures portability and compatibility across Bash environments.

Configurations: Like we export our Java and Python path to this file to be configured in every terminal as given below.

```bash
# Set Java path
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH

# Set Python path
export PYTHON_HOME=/usr/local/bin/python3
export PATH=$PYTHON_HOME:$PATH
```

Example: Let's add welcome lines to .bashrc file as below. To open it execute ***nano ~/.bashrc.***

```
  GNU nano 4.8                    /home/bilal/.bashrc                    Modified
export LIBGL_ALWAYS_SOFTWARE=1
source /opt/ros/noetic/setup.bash
source ~/catkin/devel/setup.bash
export ROS_MASTER_URI=http://192.168.206.4:11311
export ROS_HOSTNAME=192.168.206.4
JAVA_HOME=/usr/lib/jvm/java-1.8.0-openjdk-amd64
PATH=$PATH:$HOME/bin:$JAVA_HOME/bin
export JAVA_HOME
export JRE_HOME
export PATH

echo " ------------- Welcome Dear Bilal Ahmed --------------"
echo " ----------- Your have edited bashrc file ------------"
```

Source: We will need to the source the .bashrc file to apply the changes in the current terminal with command; ***source ~/.bashrc***.

Output:

```
bilal@Bilal: ~
bilal@Bilal:~$ source ~/.bashrc
 ------------- Welcome Dear Bilal Ahmed --------------
 ----------- Your have edited bashrc file ------------
bilal@Bilal:~$
```

Output on new terminal: Here we can see that when we opened the terminal it displayed this message.

```
bilal@Bilal: ~
 ------------- Welcome Dear Bilal Ahmed --------------
 ----------- Your have edited bashrc file ------------
bilal@Bilal:~$
```

Exercise:

Task 1: Write a script which creates a directory with your name, create 3 files in that folder, give read, write and execution permission to first, second and third file respectively for all users and list files with their new permissions.

Task 2: Write a script which takes input of Name, Degree, CMS-ID and semester, and print it systematically.

Task 3: Write a script which takes input a directory and prints all python files with info messages.

Task 4: Run python, write a script which gets the PID of Python and kills it and print info messages.